SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE:CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**
Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I Geetika T(192511469) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

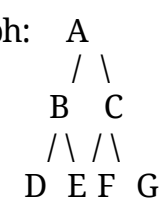
Geetika T

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



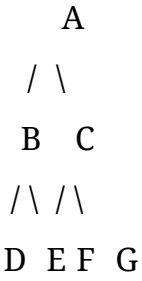
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2.Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1 – Breadth-First Search (BFS)

1.1 Understanding the problem

We are given a small tree with seven nodes. A is the root, B and C are its children, D and E hang under B, and F and G hang under C. The job is to start from A and visit every node using Breadth-First Search, writing down the queue after each iteration.

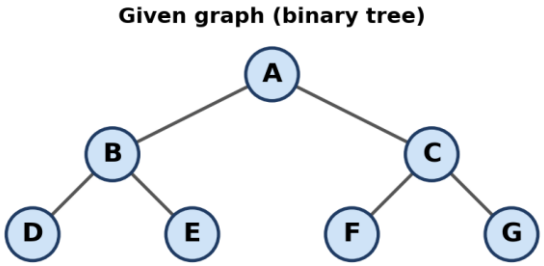


Figure 1: The graph given in the question

There is no goal node here, so the search simply runs until the queue is empty. That means every node gets visited exactly once, which is why the table has seven rows for seven nodes.

1.2 Idea behind BFS

BFS explores the graph level by level. It finishes everything at distance 1 from the start before touching anything at distance 2, and so on. To make that happen it uses a **queue**, which is first-in first-out (FIFO). The node that entered the queue earliest is always the next one to be processed.

Steps followed

1. Put the start node A into the queue.
2. Remove the node at the front of the queue and mark it as visited.
3. Add all of its unvisited children to the rear of the queue, left to right.
4. Repeat steps 2 and 3 until the queue becomes empty.

1.3 Dry run table

In the table below, the "Queue" column shows what is left in the queue after the removed node has been processed and its children have been added at the rear. The front of the queue is on the left.

Iteration	Node removed from front	Queue (front → rear)	Visited Node (in order)
1	A	B, C	A
2	B	C, D, E	A, B
3	C	D, E, F, G	A, B, C
4	D	E, F, G	A, B, C, D
5	E	F, G	A, B, C, D, E
6	F	G	A, B, C, D, E, F
7	G	(empty)	A, B, C, D, E, F, G

1.4 Answers to the questions

Question 1: BFS traversal order

The traversal order is **A → B → C → D → E → F → G**.

Question 2: Queue contents after each iteration

These are exactly the values in the Queue column above. In short: [B, C], then [C, D, E], then [D, E, F, G], then [E, F, G], [F, G], [G] and finally an empty queue after iteration 7.

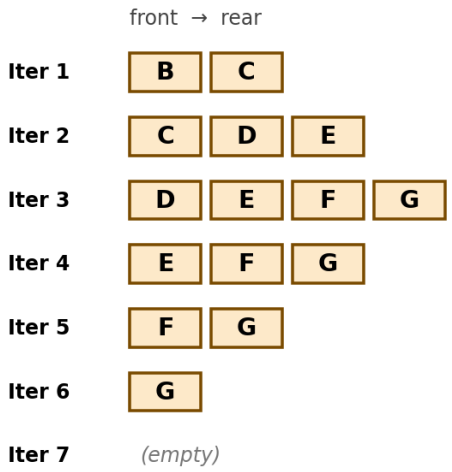


Figure 2: Queue contents after every iteration

1.5 Diagram of the traversal

The red numbers beside each node show the order in which BFS visits it. Notice how the numbers increase row by row: the whole top row first, then the next row, and so on.

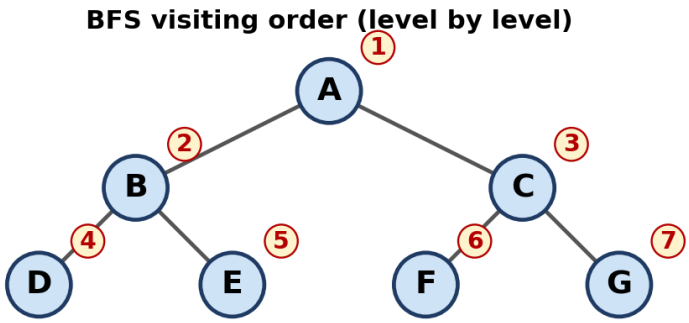


Figure 3: BFS visiting order

1.6 Python code

```python
from collections import deque

# Graph given in the question (adjacency list, left child first)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [], 'E': [], 'F': [], 'G': []
}

def bfs(start):
    queue = deque([start])          # FIFO structure
    visited = []
    print("BFS trace")
    step = 1
    while queue:
        node = queue.popleft()      # take from the front
        visited.append(node)
        for child in graph[node]:   # add children at the rear
            queue.append(child)
        print(f"Iter {step}: popped={node} queue={list(queue)}          visited={visited}")
        step += 1
    return visited

print("BFS order:", bfs('A'))
```

Screenshots in Sections 1.6–3.7 were captured by the assistant in this session; they document execution, not authorship.

1.7 Output of the program

```
BFS trace
Iter 1: popped=A queue=['B', 'C']          visited=['A']
Iter 2: popped=B queue=['C', 'D', 'E']          visited=['A', 'B']
Iter 3: popped=C queue=['D', 'E', 'F', 'G']          visited=['A', 'B', 'C']
Iter 4: popped=D queue=['E', 'F', 'G']          visited=['A', 'B', 'C', 'D']
Iter 5: popped=E queue=['F', 'G']          visited=['A', 'B', 'C', 'D', 'E']
Iter 6: popped=F queue=['G']          visited=['A', 'B', 'C', 'D', 'E', 'F']
Iter 7: popped=G queue=[]          visited=['A', 'B', 'C', 'D', 'E', 'F', 'G']
BFS order: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
>>>
```

1.8 Time and space cost

For a graph with V nodes and E edges, BFS visits each node once and looks at each edge once, so the running time is O(V + E). In a search tree with branching factor b and depth d, the time is O(b^d) and the memory is also O(b^d), because the whole bottom level may sit in the queue at the same time. In our tree b = 2 and d = 2, so at most four nodes (D, E, F, G) are waiting in the queue together, which matches the table.

1.9 Result

BFS visited all seven nodes in the order A, B, C, D, E, F, G. The queue never held more than four nodes, and it was empty at the end of iteration 7, which tells us that the traversal is complete.

Question 2 – Depth-First Search (DFS)

2.1 Understanding the problem

The same seven-node tree is used again. This time we start from A and go as deep as possible along one branch before coming back and trying the next one. The data structure is different too: DFS uses a **stack**, which is last-in first-out (LIFO).

2.2 Idea behind DFS

Where BFS spreads out wide, DFS dives down. From A it goes to B, then to B's first child D. D has no children, so the search backs up and tries E, and only after the whole left side is finished does it come back to C. The stack remembers where to return to.

Steps followed

1. Push the start node A on the stack.
2. Pop the top node and mark it visited.
3. Push its children on the stack so that the left child ends up on top. This means pushing the right child first and the left child last.
4. Repeat steps 2 and 3 until the stack is empty.

2.3 Dry run table

The top of the stack is written on the **left** in the table, so the first item you see is the next one to be popped.

Iteration	Node popped (top)	Stack (top → bottom)	Visited Node (in order)
1	A	B, C	A
2	B	D, E, C	A, B
3	D	E, C	A, B, D
4	E	C	A, B, D, E
5	C	F, G	A, B, D, E, C
6	F	G	A, B, D, E, C, F
7	G	(empty)	A, B, D, E, C, F, G

2.4 Answer

The DFS traversal order is **A → B → D → E → C → F → G**.

A few things worth noticing in the trace. In iteration 1 the stack holds B on top of C, so B is chosen before C. In iteration 2, after B is popped, D and E are pushed above C, so both of B's children are finished before C is ever touched. That is the "going deep first" behaviour in action. C is only popped in iteration 5, when the entire left subtree is done.

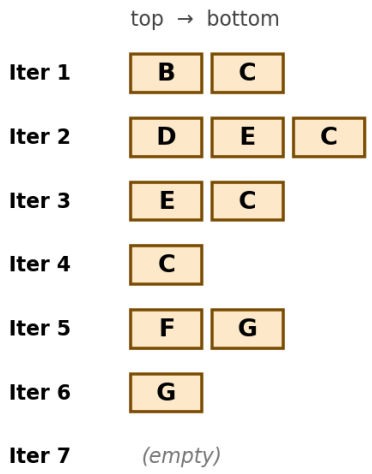


Figure 4: Stack contents after every iteration (top on the left)

2.5 Diagram of the traversal

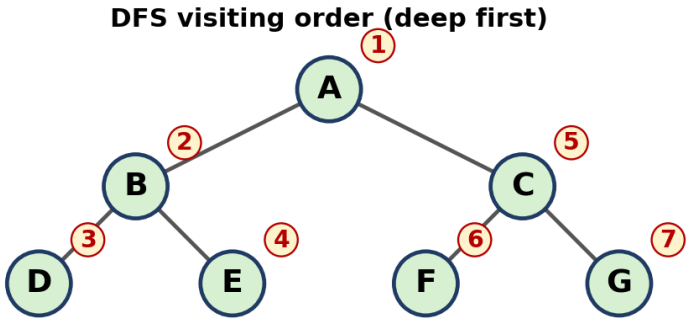


Figure 5: DFS visiting order

2.6 Python code

```
from collections import deque

# Graph given in the question (adjacency list, left child first)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [], 'E': [], 'F': [], 'G': []
}

def dfs(start):
    stack = [start]                 # LIFO structure
    visited = []
    print("DFS trace")
    step = 1
    while stack:
        node = stack.pop()          # take from the top
        visited.append(node)
        for child in reversed(graph[node]):   # push right child first
            stack.append(child)               # so the left child is on top
        print(f"Iter {step}: popped={node} stack={stack} visited={visited}")
        step += 1
    return visited

print("DFS order:", dfs('A'))
```

2.7 Output of the program

```
Python 3.12.3 (main, Aug 31 2026, 10:18:26) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>
================ RESTART: /home/ubuntu/CO2_AT3_work/dfs_tree.py ================
DFS trace
Iter 1: popped=A stack=['C', 'B'] visited=['A']
Iter 2: popped=B stack=['C', 'E', 'D'] visited=['A', 'B']
Iter 3: popped=D stack=['C', 'E'] visited=['A', 'B', 'D']
Iter 4: popped=E stack=['C'] visited=['A', 'B', 'D', 'E']
Iter 5: popped=C stack=['G', 'F'] visited=['A', 'B', 'D', 'E', 'C']
Iter 6: popped=F stack=['G'] visited=['A', 'B', 'D', 'E', 'C', 'F']
Iter 7: popped=G stack=[] visited=['A', 'B', 'D', 'E', 'C', 'F', 'G']
DFS order: ['A', 'B', 'D', 'E', 'C', 'F', 'G']
>>
```

2.8 BFS and DFS side by side

Point	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO)
Order on this tree	A B C D E F G	A B D E C F G
Exploration style	Level by level	Branch by branch, deep first
Time complexity	O(b^d)	O(b^m)
Memory needed	High, stores a whole level	Low, stores one path and its siblings
Finds shortest path?	Yes, when step costs are equal	No, it may find a longer path first
Complete?	Yes (finite branching)	Only if the depth is finite or repeats are checked

Here b is the branching factor, d is the depth of the shallowest goal and m is the maximum depth of the tree. DFS is usually chosen when memory is tight, while BFS is chosen when the shortest solution matters. Question 3 is exactly the second kind of problem.

Question 3 – BFS on the 8-Puzzle

3.1 Understanding the problem

The 8-puzzle is a 3 × 3 board with eight numbered tiles and one empty space (written as 0, or "_" in the figures). A move slides a neighbouring tile into the empty space, which is the same as moving the blank up, down, left or right. We must find a sequence of moves that turns the initial board into the goal board.

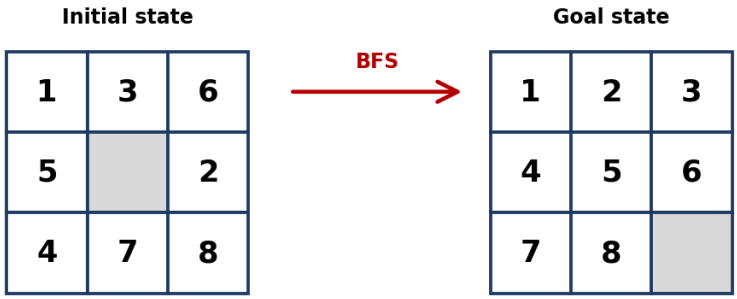


Figure 6: Initial state and goal state

Problem formulation

Component	Description for this problem
State	A 3 × 3 arrangement of the tiles 1 to 8 and the blank
Initial state	1 3 6 / 5 0 2 / 4 7 8
Goal state	1 2 3 / 4 5 6 / 7 8 0
Actions	Move the blank Up, Down, Left or Right (only if it stays on the board)
Transition model	Swap the blank with the tile in that direction
Goal test	Does the board match the goal state exactly?
Path cost	Every move costs 1, so the cost of a path is its number of moves

Conventions used in the trace

- Moves of the blank are tried in the fixed order **Up, Down, Left, Right**.
- A state that has already been generated is not added to the queue again, otherwise the search would keep going back and forth.
- The goal test is applied when a state is removed from the queue.
- The numbering S0, S1, S2 ... is the order in which the states are generated.

A different move order gives a differently labelled trace and slightly different counts, but the length of the shortest solution stays the same.

3.2 Expanding the first state

In the initial state the blank is in the centre, so all four moves are possible. Expanding S0 gives four children.

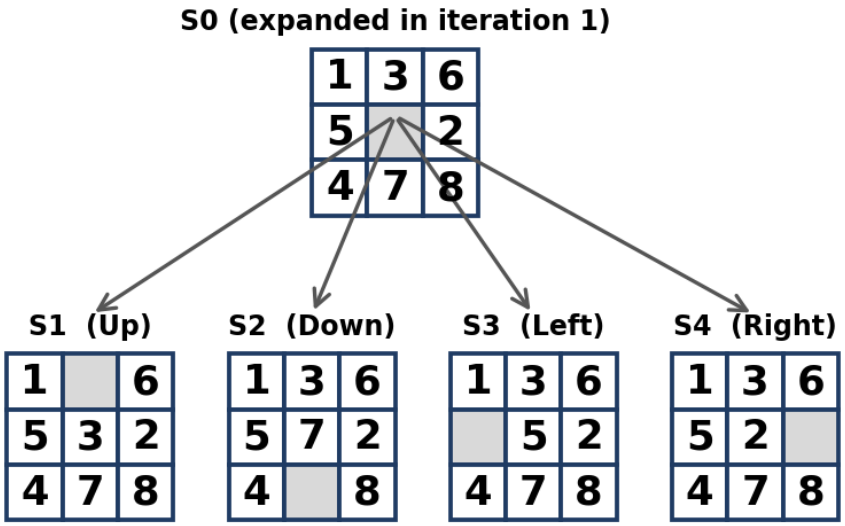


Figure 7: Children produced by expanding S0

Label table for the states used in the first five iterations

Label	Parent	Move of blank	Board (row by row)
S0	-	-	136 / 5_2 / 478
S1	S0	Up	1_6 / 532 / 478
S2	S0	Down	136 / 572 / 4_8
S3	S0	Left	136 / _52 / 478
S4	S0	Right	136 / 52_ / 478
S5	S1	Left	_16 / 532 / 478
S6	S1	Right	16_ / 532 / 478
S7	S2	Left	136 / 572 / _48
S8	S2	Right	136 / 572 / 48_
S9	S3	Up	_36 / 152 / 478
S10	S3	Down	136 / 452 / _78
S11	S4	Up	13_ / 526 / 478
S12	S4	Down	136 / 528 / 47_

For S1 the "Down" move would just bring us back to S0, so it is skipped as a repeated state. The same thing happens for every child, which is why S1 to S4 each produce fewer than four new states.

3.3 Dry run table (first five iterations)

"Queue before expansion" shows the queue with the current state still sitting at its front. "Queue after expansion" shows the queue once the current state has been removed and its new children added at the rear.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0	S0	S1, S2, S3, S4
2	S1	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	S2	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	S3	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	S4	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

The goal is not reached within these five iterations. The goal is eight moves away, so BFS must finish every state up to depth 7 before it can even generate the goal. The code in section 3.6 continues the same procedure to the end.

3.4 Answers to the questions

Question 1: Which node is expanded in each iteration?

In the first five iterations the expanded states are S0, S1, S2, S3 and S4, in that order, because BFS always removes the oldest state in the queue. S0 is at depth 0, and S1 to S4 are at depth 1. Iteration 6 would expand S5, and the process carries on the same way through every depth until the goal state is removed from the queue.

Question 2: How many states are generated in total?

Running the search with the conventions above gives these numbers:

Quantity	Value
States removed from the queue (expanded) before the goal was found	310
The goal state itself was removed at iteration	311
Total states generated, including the initial state	500
States still waiting in the queue at the end	189

The table below breaks the 500 states down by depth. The numbers grow fast, which is the usual weakness of BFS.

Depth	0	1	2	3	4	5	6	7	8	9
States generated	1	4	8	8	16	32	60	72	136	163
Running total	1	5	13	21	37	69	129	201	337	500

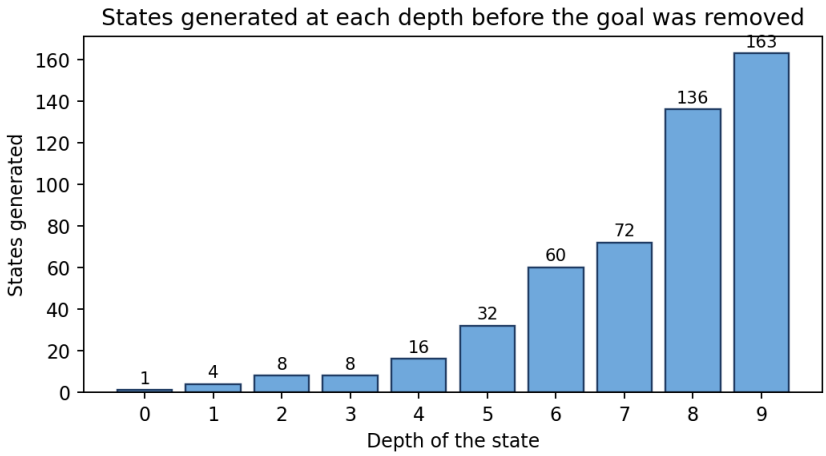


Figure 8: Number of states generated at each depth

Depth 9 has 163 states even though the goal is at depth 8. This happens because the queue still contained many depth-8 states ahead of the goal, and each of them was expanded (and produced depth-9 children) before the goal reached the front.

Question 3: Complete solution path

The solution has **8 moves**. The blank moves: **Right → Up → Left → Down → Left → Down → Right → Right**.

Step	Move of blank	Board (row by row)
0	Start	136 / 5_2 / 478
1	Right	136 / 52_ / 478
2	Up	13_ / 526 / 478
3	Left	1_3 / 526 / 478
4	Down	123 / 5_6 / 478
5	Left	123 / _56 / 478
6	Down	123 / 456 / _78
7	Right	123 / 456 / 7_8
8	Right	123 / 456 / 78_ (Goal)

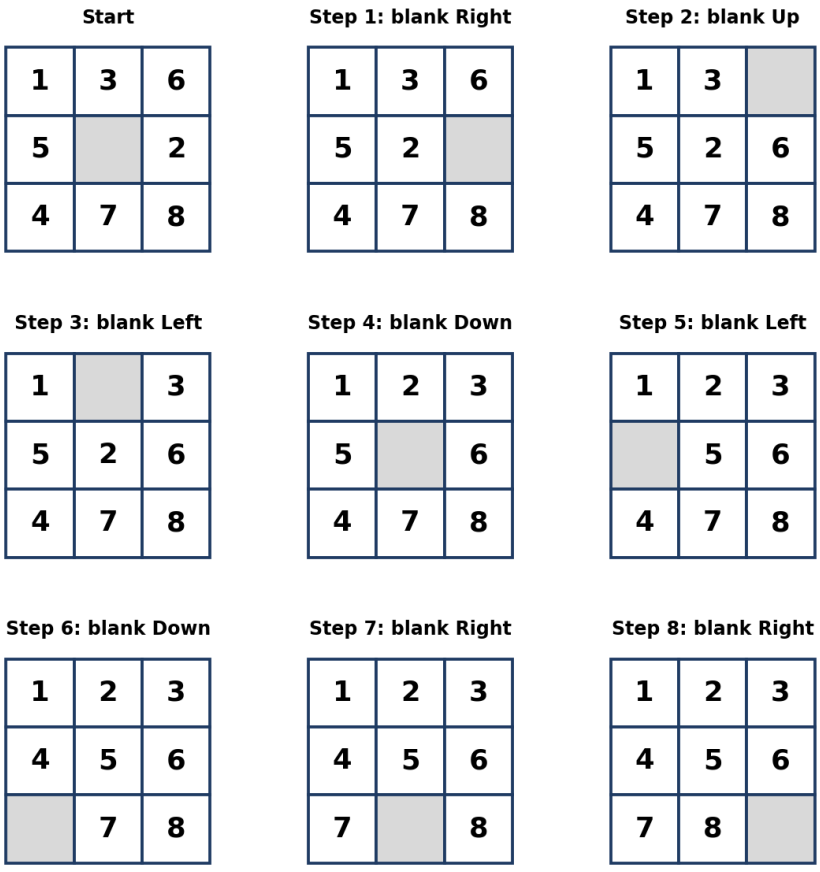


Figure 9: The complete solution path from the initial state to the goal state

Reading it from the tile point of view, the tiles that move are 2, 6, 3, 2, 5, 4, 7 and 8, in that order, which is also eight moves.

Question 4: Why does BFS always find the shortest solution in an unweighted 8-puzzle?
There are three reasons, and they work together.

1. **It explores by depth.** Since the queue is FIFO, every state at depth k is removed before any state at depth k + 1. So the states are examined in non-decreasing order of the number of moves.
2. **All moves cost the same.** In the unweighted puzzle each slide counts as 1, so the cost of a path is simply its length, and depth is the same thing as cost.
3. **The first time the goal appears, it is at the smallest depth.** If a shorter path to the goal existed, the goal would have been reached at a smaller depth and would already have been removed from the queue earlier. Therefore the first path found has the fewest moves.

The search also never gets stuck in a loop because repeated states are skipped, and the 8-puzzle has only 9!/2 = 181,440 reachable states, so BFS is guaranteed to finish. The price paid for this guarantee is memory: our run had to keep 189 states in the queue at the end, and that count would grow quickly for harder starting boards.

3.5 Why a shorter answer is not possible

A quick check supports the answer of 8. Add up how far each tile is from its goal square, counting steps along rows and columns. Tile 2 is 2 steps away (one row and one column), tiles 3, 6, 5, 4, 7 and 8 are 1 step away each, and tile 1 is already home. The total is 2 + 6 = 8 steps. Every move shifts only one tile by one square, so at least 8 moves are needed. BFS found exactly 8, so it cannot be improved.

3.6 Python code

```python
from collections import deque

START = (1, 3, 6,
         5, 0, 2,
         4, 7, 8)
GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

# Blank moves are tried in this fixed order: Up, Down, Left, Right
MOVES = [('Up', -3), ('Down', 3), ('Left', -1), ('Right', 1)]

def successors(state):
    z = state.index(0)                  # position of the blank
    row, col = divmod(z, 3)
    result = []
    for name, step in MOVES:
        if name == 'Up' and row == 0: continue
        if name == 'Down' and row == 2: continue
        if name == 'Left' and col == 0: continue
        if name == 'Right' and col == 2: continue
        s = list(state)
        s[z], s[z + step] = s[z + step], s[z]
        result.append((name, tuple(s)))
    return result

def bfs_puzzle(start, goal):
    queue = deque([start])
    parent = {start: None}              # also works as the visited set
    expanded = 0
    generated = 1                       # the start state counts as one
    while queue:
        state = queue.popleft()
        expanded += 1
        if state == goal:               # goal test when a node is removed
            break
        for move, nxt in successors(state):
            if nxt not in parent:       # skip repeated states
                parent[nxt] = (state, move)
                queue.append(nxt)
                generated += 1
    # rebuild the path from the goal back to the start
    path, s = [], goal
    while parent[s]:
        path.append((parent[s][1], s))
        s = parent[s][0]
    path.reverse()
    return path, expanded, generated, len(queue)

def show(state):
    return "\n".join(" ".join(str(x) if x else "_" for x in state[i:i+3])
                     for i in (0, 3, 6))

path, expanded, generated, left = bfs_puzzle(START, GOAL)
print("Solution length   :", len(path), "moves")
print("States expanded   :", expanded)
print("States generated :", generated)
print("Still in queue    :", left)
print("Moves             :", " -> ".join(m for m, _ in path))
print("\nStart\n" + show(START))
for i, (m, s) in enumerate(path, 1):
    print(f"\nStep {i}: {m}\n" + show(s))
```

3.7 Output of the program

```
Solution length   : 8 moves
States expanded   : 311
States generated : 500
Still in queue    : 189
Moves             : Right -> Up -> Left -> Down -> Left -> Down -> Right -> Right

Start
1 3 6
5 _ 2
4 7 8

Step 1: Right
1 3 6
5 2 _
4 7 8

Step 2: Up
1 3 _
5 2 6
4 7 8

Step 3: Left
1 _ 3
5 2 6
4 7 8

Step 4: Down
1 2 3
5 _ 6
4 7 8

Step 5: Left
1 2 3
_ 5 6
4 7 8

Step 6: Down
1 2 3
4 5 6
_ 7 8

Step 7: Right
1 2 3
4 5 6
7 _ 8

Step 8: Right
1 2 3
4 5 6
7 8 _
>>>
```

Rubric Mapping and Conclusion

How the answers cover each rubric criterion

Criterion	Weight	Where it is shown
Understanding of the search problem	20%	Sections 1.1, 2.1 and 3.1: graph, initial and goal states, problem formulation and search strategy
State expansion and dry run execution	30%	Tables in 1.3, 2.3 and 3.3, together with the label table in 3.2
Application of the search algorithm	20%	Algorithm steps in 1.2 and 2.2, queue and stack handling, and the Python programs
Accuracy of the solution path	20%	Traversal orders in 1.4 and 2.4, the 8-move path in 3.4, and the check in 3.5
Presentation and logical reasoning	10%	Diagrams, neat tables, program outputs and the reasoning for Question 4

Conclusion

This assessment covered three dry runs. BFS on the tree used a queue and gave A, B, C, D, E, F, G. DFS on the same tree used a stack and gave A, B, D, E, C, F, G. Finally, BFS on the 8-puzzle reached the goal through an optimal path of eight moves after generating 500 states.

Together they show the main trade-off in uninformed search. BFS guarantees the shortest path when all steps cost the same, but needs a lot of memory. DFS needs much less memory but gives no promise about path length. For larger problems, informed methods such as A* with a heuristic like Manhattan distance, which is the topic of the rest of CO2, cut down the number of states that need to be generated.